What Does a Streaming Client See When an LLM Server Crashes?

Client-Visible Token Anomalies in vLLM and SGLang, an Exactly-Once Gateway, and KV Checkpoints

Rajshekar Medipally
Independent researcher
https://github.com/rmedipallycic/llm-inference-fault-tolerance

October 2026 (version 3)

Abstract

LLM inference servers stream tokens to clients over requests lasting seconds to minutes; when a serving process fails mid-generation, the recovery decides what the client finally shows a user or hands to an agent. With an open-source fault-injection harness that SIGKILLs unmodified vLLM and SGLang servers (Llama 3.1 8B, A10G GPUs), we report three results. First, retry is not replay: in both engines' default configurations the failure-free output was not unique at temperature 0, so naive retry contradicted text already shown in 25 of 30 vLLM and 24 of 30 SGLang trials; each engine's deterministic mode with recompute made recovery exact, at 0.46–0.57× the default throughput under load. Second, a gateway that numbers output positions, logs them before release and resumes from the logged token ids delivered every position exactly once, under stated assumptions, in all 190 single-request failures and all 360 requests of failures with 4–32 requests in flight, and removed text the engine repeated after a resume (64 times), which would otherwise have duplicated what the client showed. Third, recovery time depends on where state is kept: a standby replica cut recovery from minutes to 0.05–0.08 s for a short prompt, and vLLM's own KV offloading to storage the standby reads cut it from 5.14 s to 0.64 s at 16K prompt tokens, and from a median of 4.86 s to 1.48 s with 32 requests in flight, for under 1% decoding cost and 5–8% later first tokens. A simulation driven by the measurements found no benefit from adaptive checkpoint policies. All raw logs and table-generating scripts are public.

1 Introduction

LLM serving has become stateful streaming infrastructure. A generation request holds a growing key-value (KV) cache on an accelerator and emits tokens to the client incrementally. Failure handling, however, is still largely stateless: if the serving process dies, the client or a load balancer resubmits the request and generation starts again.

That recovery has a cost that server-side metrics miss. A client that has already displayed 128 tokens may receive those tokens a second time, may receive a continuation that disagrees with what it already showed, or may lose text at the boundary. For a chat interface this is a visible glitch. For an agent that has already acted on a partial tool call or a partial plan, it is a correctness problem. Stream processing systems name this property *delivery semantics* and engineer exactly-once output deliberately [5, 2, 1]. LLM serving has no equivalent definition, and to our knowledge no measurement of what clients actually receive.

A common intuition is that at temperature 0 a retry is harmless: greedy decoding is deterministic, so resubmitting the prompt reproduces the same text and the client only has to drop the duplicate. Our measurements on vLLM refute that intuition, and so do our measurements on SGLang, for a different reason. In vLLM the output for a fixed prompt depended on what the server had served earlier; in SGLang the first request after each server start varied from start to start. Either way, the restarted server that answers a retry does not reproduce the text the client was already shown.

This paper makes four contributions:

1. **Client-visible anomalies, defined and measured.** Token-level delivery semantics for streamed generation, with anomalies defined against *two* failure-free references (Section 3); a fault-injection harness for any OpenAI-compatible server (Section 4); and a study of naive retry and recompute on vLLM and SGLang with cache ablations, deterministic modes and six prompts (Sections 5–7), which finds that in both engines' default configurations failure-free output depends on server state, so naive retry contradicts text already shown.

2. **A gateway protocol** in front of unmodified engines that numbers and logs output positions before releasing them and resumes a failed request from

the logged token ids, with its guarantee stated precisely: what it covers (positions; text derived from token ids), and what it assumes (fail-stop crashes, a surviving log, a client that remembers its position) (Section 8).

3. **Recovery and determinism, evaluated.** 190 injected failures through the gateway on both engines, including failover between two machines, where exact text needed matching compiled state in our runs (Section 9); the cost of the deterministic modes under concurrent load (Section 10); and failures with up to 32 requests in flight (Section 13).

4. **KV-checkpoint analysis.** Measured recompute and KV-copy costs from 512 to 32,768 tokens (Section 11); a simulation of checkpoint policies judged by a decision rule written before it ran (Section 12); and vLLM's built-in KV offloading used as a live checkpoint, measured while serving and through failover (Section 13).

The first part of the study (Sections 5–7) deliberately uses unmodified engines and client-side recovery only, and establishes the baseline. The second part (Sections 8–9) asks what a server-side protocol in front of the same engines achieves against it. The third part (Sections 10–13) measures what the fixes and the gateway cost under load and how much a KV-cache checkpoint shortens recovery. This is version 3 of the paper; version 1 contained the first part, version 2 added the second.

2 Background

Autoregressive serving. A request runs a prefill phase, which processes the prompt in parallel and builds the KV cache, followed by a decode phase that produces one token per step and streams it to the client. Engines such as vLLM [7] and SGLang [16] batch requests at iteration granularity [15] and manage the KV cache in fixed-size blocks.

The state lost on failure. The KV cache is a deterministic function of the weights and the token sequence, so it can always be rebuilt by prefill, up to numerical differences between prefill and decode kernels. At temperature above zero the continuation also depends on random draws. Whether those can be reproduced after a restart depends on the engine: a sampler that keeps a running random state loses it with the process, while one that derives each draw from the request's seed and the token's position, as vLLM 0.31 does (Section 9), can repeat it.

Rollback-recovery vocabulary. Classic rollback-recovery distinguishes checkpointing from logging and defines *output commit*: a message to the outside world must not be released until the state that produced it is recoverable [5]. Mapped onto LLM serving, the token sequence is a replayable input log, the KV cache is derived operator state, and the streaming client is the external sink where delivery semantics are decided.

3 Delivery model and anomalies

A failure occurs after the client has received delivered text D. After recovery the client receives recovery text R. What the client finally holds depends on the strategy: **naive retry** resubmits the original prompt with the original `max_tokens`, as many client retry wrappers do, and the retry is a complete answer, so the final text is $F = R$; **recompute** resubmits the prompt concatenated with D and requests the remaining token budget, so the server rebuilds the KV cache by prefill and continues, and $F = D \cdot R$. Neither requires engine changes. When the failure occurs, D has already been shown to a user or passed to a consumer. $F = R$ for naive retry assumes the client replaces what it showed; a client that instead appends the replay after dropping its first $|D|$ characters is the case the contradiction metric below captures.

Two references. Before the trials of each run, the harness issues the request twice without failures and records both outputs: G, the first request after the server starts, and G', the second. A reproducible server has $G = G'$. We do not assume this; we measure it, and compare every final text with both.

Anomalies.

- **Contradiction** (naive retry): R does not start with D, i.e. the replay disagrees with text the client has already displayed. A client that deduplicates by dropping the first $|D|$ characters of R would then splice two different texts. Recompute cannot contradict D by construction.
- **Duplication** (naive retry): the client receives D again. Without contradiction this is full duplication, which an idempotent client can remove.
- **Divergence**: F equals neither G nor G', i.e. the failure changed the final answer relative to any failure-free run we observed.
- **Recovery latency**: wall time from the kill to the first chunk delivered after recovery, which includes server restart.

Comparison is on text, not token ids, because the OpenAI-compatible streaming API does not guarantee one token per chunk.

4 Fault-injection harness

The harness (`harness/`, about 340 lines of Python) has three parts.

Server lifecycle. The server command is started in its own process group and declared ready when `GET /v1/models` returns 200. A fault is a `SIGKILL` of the whole process group, since vLLM and SGLang spawn worker processes that otherwise keep holding GPU memory and the port. On Windows, used only for laptop smoke tests, the equivalent is `taskkill /F /T`.

Streaming client. Requests go to `/v1/completions` with `stream=true`. Every server-sent event with non-empty text is recorded as a chunk with wall-clock and monotonic timestamps. The kill fires from the chunk callback when the configured chunk count is reached, and the client keeps reading, so chunks the server had already sent before dying are recorded as in-flight rather than lost. Kill points are counted in chunks, not tokens. In the 10 vLLM trials that recorded token ids, every chunk before the kill carried exactly one token, so there the two coincide; after recovery, one chunk per trial carried 9 or 10 tokens.

Trial protocol. Each run (one engine, strategy, temperature, and kill position) starts the server, records two failure-free references G and G', then for each trial: streams the request, kills at the kill point, restarts the server and waits for readiness, issues the recovery request, and writes a JSON record containing every chunk, the kill time, the restart time, and the client view V.

Provenance. Each run records the git commit and whether the working tree was dirty, the full command line, installed versions of `vllm`, `sglang`, `torch`, and `transformers`, the GPU model, memory, and driver from `nvidia-smi`, and the full server log. Measured runs are written to `experiments/` and committed; harness smoke tests are written to a separate, git-ignored directory and are never reported. Every table in this paper is generated by one script, `analysis/make_tables.py`, from those committed files.

Validation. The harness was exercised end to end on a laptop CPU with llama.cpp and a 0.5B model before any GPU run. That check, documented in the repository as a mechanics check rather than a result, led to two changes: a duplication metric based on shared prefix fraction rather than any shared character, and the reference-reproducibility check, after two clean seeded runs on that server disagreed at temperature 0.8.

5 Methodology

Setup. Two identical AWS `g5.xlarge` instances (one NVIDIA A10G, 24 GB; 4 vCPUs each; same machine image and driver), one per engine, Llama 3.1 8B Instruct in bf16. The harness code was identical for both engines (commits `90da903` and `ddd8890` differ only outside `harness/`).

vLLM ran in its own Python 3.12 environment with default engine flags except a 4,096-token context cap (Llama 3.1's 128k default does not fit a 24 GB KV cache; every request here is far shorter). Defaults include automatic prefix caching, which the server logs confirm was on (`enable_prefix_caching=True`). The machine image ships no CUDA toolkit, so FlashInfer's sampler, which compiles at startup, was replaced by PyTorch sampling (`VLLM_USE_FLASHINFER_SAMPLER=0`); this is recorded in every run log.

SGLang could not be started directly on that image, because several of its components compile GPU code at startup (the attempts are documented in the repository). It ran instead from the official container image `lmsysorg/sglang:v0.5.21` (digest `sha256:b1259f3e...`), which includes a CUDA 13.0 toolkit, with the harness running inside the same container so that the process-group kill reaches the server. Flags were defaults except the same 4,096-token context cap and `--mem-fraction-static 0.85` (the default left no room for the KV cache on 24 GB). The server logs record what the defaults selected: FlashInfer attention and sampling, CUDA graphs, radix cache on, and deterministic inference off. The two engines therefore differ in kernels and sampler as well as in design; we compare each engine's default behaviour, not individual kernels. Versions are in Table 1.

86 measured runs, 430 trials, harness commit(s) `90da903, 9510da0, ddd8890`. In-flight chunks received after the kill: 0 at most, over all trials.

Grid. One fixed prompt (an explanation of write-ahead logging), `max_tokens`=256, per-request seed 1234. Strategy $\in$ {naive retry, recompute} $\times$ temperature $\in \{0, 0.8\}$ $\times$ kill point $\in$ {25, 128, 230} chunks (about 10%, 50%, and 90% of the budget), 5 trials each: 12 runs and 60 trials per engine, produced by `TRIALS=5 scripts/run_matrix.sh vllm` and by `scripts/sgl_grid.sh` (the same grid with the SGLang flags above). Requests are issued one at a time, so there is never more than one request in a batch.

Ablation. After the vLLM grid showed $G \neq G'$, we repeated the kill-at-128 configurations for both strategies and temperatures with automatic prefix caching disabled (`--no-enable-prefix-caching`), all else equal: 4 runs, 20 trials, labeled "no APC". For SGLang we ran the same four configurations with

Table 1: Software and hardware, from `run.json`.

Config	Engine	torch	GPU
vLLM	0.31.0	2.13.0	NVIDIA A10G (23028 MiB), driver 595.91.07
vLLM, no APC	0.31.0	2.13.0	NVIDIA A10G (23028 MiB), driver 595.91.07
vLLM, batch-inv.	0.31.0	2.13.0	NVIDIA A10G (23028 MiB), driver 595.91.07
SGLang	0.5.21	2.13.0+cu130	NVIDIA A10G (23028 MiB), driver 595.91.07
SGLang, no radix	0.5.21	2.13.0+cu130	NVIDIA A10G (23028 MiB), driver 595.91.07
SGLang, fixed seed	0.5.21	2.13.0+cu130	NVIDIA A10G (23028 MiB), driver 595.91.07
SGLang, determ.	0.5.21	2.13.0+cu130	NVIDIA A10G (23028 MiB), driver 595.91.07

its radix cache, the corresponding feature, disabled (`--disable-radix-cache`), labeled "no radix".

Follow-up runs. After the first results, a second set of runs (harness commit `9510da0`, scripts `scripts/extras_*.sh`), all at the 128-chunk kill point with 5 trials each, tested the open questions: (i) each engine's deterministic mode, vLLM's batch invariance (`VLLM_BATCH_INVARIANT=1`) and SGLang's deterministic inference (`--enable-deterministic-inference`, with `sampling_seed` 1234 in each request), on the original prompt at both temperatures; (ii) SGLang with a fixed server seed (`--random-seed 1234`); (iii) a vLLM recompute variant, *recompute (ids)*, that resubmits token ids (the prompt's ids from `/tokenize` plus the delivered tokens' ids) instead of text, and records whether re-tokenizing the delivered text would have produced the same ids; and (iv) five further prompts (a TCP explanation, a short story, a coffee recipe, processes versus threads, and the 2008 financial crisis; `harness/prompts.json`) at temperature 0, for each engine in its default and its fixed configuration (vLLM without prefix caching, SGLang deterministic). That is 54 runs and 270 trials.

Hypotheses. Stated before the runs:

H1 At temperature 0, naive retry yields full duplication in every trial and recompute yields $F = G$ in every trial.

H2 At temperature 0.8 with a fixed seed, vLLM reproduces G across clean runs; if so, naive retry still yields full duplication and recompute diverges after the kill point.

H3 Recovery latency is dominated by server restart, not by prefill of the delivered text.

6 Results

vLLM: failure-free output was not unique. In all 12 runs of the default configuration, $G \neq G'$ (Table 2). At temperature 0 the two references agreed for the first 506 characters and then took different wording ("The log is scanned from the end…" versus "The log is scanned to determine…"). With a fixed seed at temperature 0.8 they diverged at character 9. Neither is noise: every run's G was identical to every other run's G at the same temperature, and likewise for G'. The output was a deterministic function of the server's history, not of the prompt alone. Within a run, all trials after the first produced identical text, so the 5 trials per configuration are repeated observations of at most two distinct server histories, not independent samples.

Table 2: Failure-free reproducibility: first request after server start (G) versus second (G'). LCP: their common prefix in characters (range over runs). Distinct G, G': how many different texts occurred across the runs of a row. "No APC", "no radix": the engine's prefix cache disabled.

Config	T	$G{=}G'$	LCP	$\#G$	$\#G'$
vLLM	0	0/6	506	1	1
vLLM	0.8	0/6	9	1	1
vLLM, no APC	0	2/2	1228	1	1
vLLM, no APC	0.8	2/2	1219	1	1
vLLM, batch-inv.	0	2/2	1228	1	1
vLLM, batch-inv.	0.8	2/2	1225	1	1
SGLang	0	0/6	267–676	4	1
SGLang	0.8	0/6	1–2	6	6
SGLang, no radix	0	0/2	267–412	2	1
SGLang, no radix	0.8	0/2	0–1	2	2
SGLang, fixed seed	0	0/2	267–554	2	2
SGLang, fixed seed	0.8	0/2	1	2	2
SGLang, determ.	0	2/2	1242	1	1
SGLang, determ.	0.8	0/2	3	1	1

vLLM: naive retry contradicted the client. The restarted server is always in the state that produced G, and the replay equaled G in all 30 naive-retry trials (Table 3). The text the client had already shown, D, came from a warm server and followed G'. Wherever the kill fell after the point at which G and G' part, the replay therefore contradicted D: in 25 of 30 trials, at character 506 at temperature 0 and at character 9 at temperature 0.8. The only clean case was temperature 0 with the

kill at 25 chunks (113 characters, before character 506), where the replay began with D and the anomaly was pure, removable duplication. A client that drops the first $|D|$ characters of a replay, the usual deduplication, would in the other 25 trials have produced a spliced text that no failure-free run generated.

vLLM: recompute kept the shown text but changed the answer. Recompute never contradicted D, but its final text matched G in 4 of 30 trials and G' in 2 of 30. At temperature 0 with the kill at 25 chunks, all five final texts departed from both references at character 267, after the kill (113) and before G and G' part (506): a third greedy output. The same third point of departure appeared without any failure. In each recompute run, trials 1–4 ran on a server whose only earlier request was trial 0's recovery (prompt plus D), and their pre-kill streams, which no failure had yet touched, departed from both references in 16 of 24 trials. The two $F{=}G'$ cases are trial 0 at the 230-chunk kill; the four $F{=}G$ cases are trials 1–4 at temperature 0.8 with the kill at 25 chunks, where a 25-token recovery prefix left the continuation on the cold-server path.

vLLM hypotheses. H1 is refuted: at temperature 0, naive retry gave full duplication only at the earliest kill point (5 of 15 trials) and recompute never reproduced G (0 of 15). **H2's premise is refuted**: a fixed seed did not reproduce G across requests, so its predictions cannot be tested as stated. **H3 is supported** (Table 4): restart took 108–195 s and the first recovered token arrived 0.23–0.32 s after the server was ready. Recompute's extra prefill was visible but small: median time to first token after restart rose from 0.24 to 0.27 s at temperature 0 and from 0.28 to 0.31 s at 0.8 as the kill moved from 25 to 230 chunks, against 0.23–0.28 s for naive retry. The higher median restart for recompute is not a strategy effect: restart happens before the recovery request is sent, and both initial startup and restart rose from about 110–122 s to 160–195 s from the third trial of the temperature-0, 230-chunk recompute run onward and stayed there for the rest of the grid; the cause was not identified. In the ablation, run later the same day, restarts were back to 107–120 s.

vLLM prefix-cache ablation. With automatic prefix caching disabled (rows "no APC"), $G = G'$ in all 4 runs at both temperatures (Table 2), and naive retry no longer contradicted the client: in 10 of 10 trials the replay equaled both references and began with D, so the anomaly was pure duplication that a client can remove (Table 3). The cache setting also changed the reference itself at temperature 0: the no-APC output departs from the default configuration's G at character 267, although G is the first request after start, when the cache is empty. At temperature 0.8 the no-APC reference equals the default G exactly.

Recompute behaved differently by temperature. At 0.8 its final text equaled G in 5 of 5 trials. At temperature 0 it equaled neither reference in 5 of 5: all five departed at character 655, 43 characters after the kill (612), so recompute changed the greedy answer even with no prefix cache. That final text is character-for-character identical to the default configuration's recompute text at the 25-chunk kill. Recompute's answer at temperature 0 is therefore not explained by the cache. It is consistent with the delivered text being processed as prompt (prefill) rather than generated (decode), or with re-tokenization of D; these data do not separate the two.

SGLang: the first request after a start was not reproducible. SGLang showed a different pattern (Table 2). At temperature 0 the second request on a running server, G', was the same text in all 6 runs, but the first request after a start was not: the 21 first-after-start outputs we recorded (6 references G and 15 naive-retry replays) contained 10 distinct texts, none equal to G', departing from it at characters 267 to 676. Disabling the radix cache did not change this: G' was again the same in both runs, and 7 first-after-start outputs gave 4 distinct texts. In SGLang the output therefore depended on whether the server had just started, not on cache contents, and the cold output was not repeatable from one start to the next. We did not identify the source. One untested candidate is that, with no seed given, SGLang chooses a new server random seed at every start; the logs record a different value each time.

At temperature 0.8 the request seed had no observable effect: G and G' diverged within the first two characters in all 8 runs, and the 20 naive-retry pre-kill streams were all different. SGLang provides seeded, reproducible sampling through an opt-in deterministic mode with a seed in its own sampling parameters [11]; our harness sends the OpenAI-style `seed` and did not enable that mode, so SGLang's 0.8 results describe unseeded sampling.

SGLang: the same consequences. Naive retry contradicted text already shown in 24 of 30 default-configuration trials: at temperature 0 in 0, 4 and 5 of 5 trials as the kill moved from 25 to 128 to 230 chunks (the warm stream D followed G', the cold replay did not), and at 0.8 in 15 of 15. Without the radix cache it did so in 10 of 10. Recompute never contradicted D, and its final text matched neither failure-free output of its run in 26 of 30 default-configuration trials. Recovery was again dominated by restart (Table 4): 125–145 s with the radix cache on, and the first recovered token 0.33–0.51 s after the server was ready. Restarts

in the no-radix runs, which ran last, took 172–195 s; as with vLLM's drift, we did not identify the cause and do not attribute it to the cache setting. For SGLang, as for vLLM, **H1 is refuted** (at temperature 0 naive retry gave full duplication only at the earliest kill point, and recompute reproduced G in 3 of 15 trials), **H2's premise is refuted** (the seed had no observable effect), and **H3 is supported**.

7 Follow-up experiments

Re-tokenization is not the cause. In all 10 recompute (ids) trials, re-tokenizing the delivered text gave exactly the token ids the engine had generated (Table 5). Resubmitting the ids instead of the text changed nothing: without prefix caching, the final text again departed from the reference at character 655, the same place as text-based recompute. The most plausible remaining explanation for recompute's change of answer, which we did not test directly, is that the delivered tokens' KV cache, computed in one prefill on recovery, differs numerically from the one built token by token during decoding. Requesting token ids on every call also changed vLLM's default-configuration reference to the no-APC text, so the default-configuration ids run serves only as a re-tokenization check.

SGLang's server seed is not the cause. With `--random-seed 1234`, SGLang's first request after a start still varied from start to start (Table 2) and naive retry still contradicted the client in 5 of 5 trials at temperature 0.

Deterministic modes made recompute exact. With vLLM's batch invariance, $G = G'$ in all 4 runs, naive retry never contradicted the client (0 of 10), and recompute reproduced the failure-free output exactly in 10 of 10 trials, at temperature 0 and at 0.8 with a fixed seed (Table 3). Its temperature-0 output was identical to the no-APC output. With SGLang's deterministic inference at temperature 0, $G = G'$, naive retry never contradicted the client, and recompute was exact in 5 of 5. That is consistent with the explanation above: kernels that give the same result regardless of batch and prefill shape remove the difference between the recovered and the original computation. At temperature 0.8, SGLang's deterministic mode made each reference repeatable across runs, but G and G' still diverged at character 3; naive retry contradicted the client in 5 of 5 trials and recompute was not exact (0 of 5), so its sampling was not reproducible across requests in this configuration.

The pattern holds across prompts. Table 6 repeats the 128-chunk, temperature-0 configuration on five more prompts. In vLLM's default configuration, $G \neq G'$ for three of the five prompts, and for exactly those three naive retry contradicted the client in 5 of 5 trials (15 of 25 overall); for the other two, the first and second requests happened to agree. Without prefix caching, $G = G'$ for all five and naive retry never contradicted (0 of 25), but recompute was exact for only one prompt (5 of 25). In SGLang's default configuration $G = G'$ in only 1 of 10 runs, naive retry contradicted the client in 14 of 25 trials, and recompute was exact in 3 of 25. In SGLang's deterministic mode, all 10 runs had $G = G'$, naive retry never contradicted (0 of 25), and recompute was exact in 25 of 25.

Cost. The deterministic modes did not slow recovery much in these runs: restart times were in the same range, and the first token after restart arrived in 0.26–0.30 s with vLLM's batch invariance. With SGLang's deterministic mode it arrived in 0.34–0.46 s in some runs and 3–4 s in others, which we did not investigate. Throughput under load, which is what batch-invariant kernels are reported to cost [6], is measured in Section 10.

8 An exactly-once gateway

The measurements above leave recovery to the client, which then has to resubmit, splice and deduplicate. The gateway (`gateway/`, about 650 lines of Python with no dependencies beyond `requests`) moves that work to a process in front of unmodified engine replicas. A client uses the usual OpenAI-compatible streaming API and needs no recovery code.

Protocol. (1) *Numbering*: every streamed entry carries `seq`, the position of its first output token, and `n`, the number of tokens it holds, in an extra field of the server-sent event. (2) *Output commit*: each entry is appended to a per-request log file, optionally with `fsync`, before it is released to the client [5], so the log contains everything the client has seen. (3) *Engine failure*: a broken stream, an HTTP error, or no chunk for a stall timeout counts as a failed attempt. The gateway picks a healthy replica, trying the others before the one that failed, and resubmits the prompt's token ids followed by the ids of every committed token, asking only for the remaining budget. Nothing is re-tokenized. This is the recompute strategy of Section 3, done once, on the server side. (4) *Client reconnect*: a client that lost its connection asks for the request's stream from a given position and receives exactly the entries it has not seen. (5) *Gateway restart*: on start, the gateway resumes every logged request that has no completion record.

What it guarantees, and under which assumptions. The failure model is fail-stop crashes of engine and gateway processes; the log directory survives; and a client that loses its connection remembers the position after the last entry it consumed. Under these assumptions:

- Each output position is committed to the log exactly once and in order, and a client that reconnects from its own last consumed position receives every remaining position exactly once, in order, unless the request fails for good. The gateway does not know what a client received, only what it committed: an entry written and sent but lost in transit is sent again only because the client asks from its own position. The guarantee therefore reaches an application end to end only if the client keeps that position durably together with what it consumed; acknowledged delivery is not part of the protocol.
- A *position* is a token with vLLM (ids mode) and a stream chunk with SGLang (text mode, below), so the SGLang guarantee is per chunk, and its recovery re-tokenizes the delivered text.
- The continuation never contradicts delivered text, because the delivered tokens are its context. It reproduces the failure-free text only where the engine does (Section 7).
- In ids mode, the text of the first entries after a failure, and of the first entries of a request whose prompt the client sent as token ids, is checked against the decoding of the token ids, with the prompt as context; text the engine repeated is removed. The text a client concatenates then equals what the delivered ids add to the prompt's decoding. This was added in version 3, after the engine's text was found to repeat delivered text (Section 13).

Resume modes. With vLLM the gateway uses token ids (`/tokenize` for the prompt, token ids in the stream for the output), and `seq` counts tokens. SGLang's OpenAI-compatible stream did not return token ids in the form the gateway reads, so with SGLang it resubmits the prompt text plus the delivered text and `seq` counts chunks; the re-tokenization check of Section 7 was done on vLLM only.

Tests. Twelve tests (`tests/`) run the gateway and a mock engine as separate processes and SIGKILL them mid-stream: engine restart, failover, two failures in one request, temperature 0.8, text mode, client reconnect, gateway restart, and a negative control in which naive retry without the gateway repeats positions and the checker flags it. Version 3 adds the fixes found during the runs of Section 13, each with a test that fails on the code before it: the event encoding, the text of resumed entries (repeated text, special tokens, characters split across the prompt and the output), and a killed engine's stream cut in the middle of a chunk; and a test with 64 clients connecting at once (which also passed before its fix, on one machine); 22 tests in all.

9 Gateway evaluation

Setup. The same instance type, machine image, model and prompt as above; vLLM 0.31.0 and SGLang 0.5.21, SGLang again inside the same container image; `max_tokens`=256, seed 1234, 10 trials per run, requests one at a time. *Restart*: one replica; the replica serving the request is SIGKILLed once the given number of positions has reached the client, and is restarted; the gateway waits for it. Temperature 0 and 0.8, kill after 25, 128 and 230 positions, plus each engine's deterministic mode at temperature 0 and 128 (`scripts/run_gateway_matrix.sh`). *Failover*: two machines of the same type, same image, driver and vLLM version, with the gateway and replica 0 on the first and replica 1 on the second; replica 0 is killed after 128 positions and the gateway resumes on replica 1 (`scripts/run_failover_two_hosts.sh`). *Overhead*: 30 failure-free requests directly to the engine and 30 through the gateway, alternating, without and with `fsync`. Delivery is checked from the positions each client recorded (`gateway/check.py`); "same text" means the client's final text equals one of the run's two failure-free references. Every table here is generated by `analysis/make_tables.py`.

How times through the gateway are measured. Version 2 of this paper took recovery latency and time to first token through the gateway at the client. During the runs of Section 13 we found that the measuring client read the gateway's stream with the Python `requests` library's `iter_lines`, which waits for 512 bytes on a stream without chunked transfer encoding, as the gateway's is; the client therefore saw each event only when the next one arrived. Direct requests to the engines, whose streams are chunked, were not affected. All times through the gateway reported here are taken instead from the gateway's log, which stamps each entry when it is appended, just before it is written and released (`analysis/gw_timing.py`); the client has since been fixed. This changes the failover and overhead figures of version 2, by up to about 0.1 s and about 30 ms respectively, and nothing else.

Delivery. In all 190 kill trials, 140 with restart and 50 with failover, the client received every position ex-

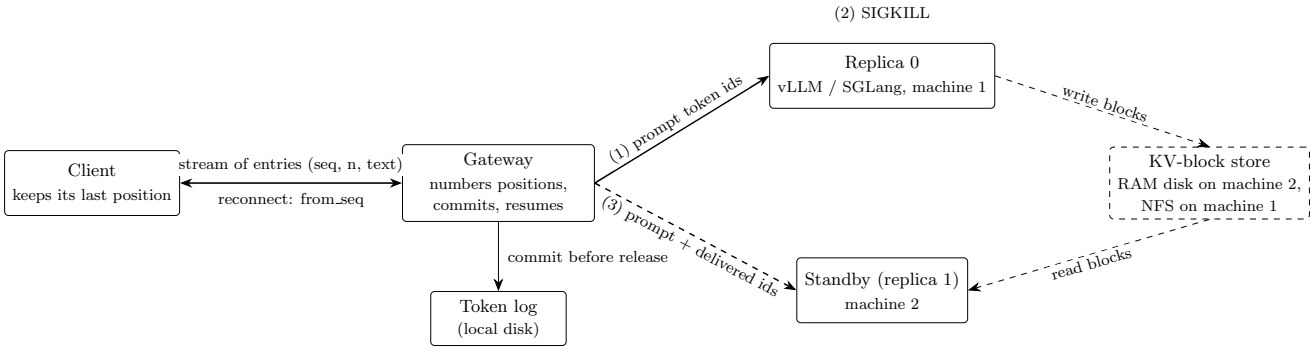


Figure 1: The serving path evaluated in this paper. The gateway numbers every output position and commits each entry to its log before releasing it to the client; a client that reconnects names the position it needs. When the serving replica fails (2), the gateway resumes the request on a restarted replica or on the standby (3) with the prompt and the delivered token ids. In Section 13 (dashed store), both replicas also offload KV-cache blocks to a store the standby can read, so it loads them instead of recomputing them.

actly once: 0 duplicated and 0 missing positions, every request complete, one recovery attempt each (Tables 7 and 8). Without the gateway, naive retry contradicted the client in 25 of 30 (vLLM) and 24 of 30 (SGLang) trials of the corresponding configurations (Section 6).

Same text after restart. With the deterministic modes the recovered text equalled the failure-free text in 10/10 (vLLM) and 10/10 (SGLang) trials, as client-side recompute did in Section 7. In the default modes at temperature 0 it did so in 2/30 and 3/30 trials, where the two references themselves differed. At temperature 0.8 SGLang matched in 0/30 trials: its 30 recovered texts all differed from one another, consistent with the request seed having no effect (Section 6). vLLM matched in 11/30, and more often the later the kill. That is explained by how vLLM 0.31 samples. Its model runner, which the server logs show was in use, picks a token by adding noise to the logits and taking the maximum, and for a request with an explicit seed it always draws the noise from a hash of the seed, the token's absolute position in the sequence, and the token id, keeping no running random state (`vllm/v1/worker/gpu/sample/gumbel.py` and `sampler.py`). A resumed request, whose tokens occupy the same positions, therefore draws the same noise, and its sample differs from the original only where the logits differ; the later the kill, the fewer tokens remain in which they can. Without an explicit seed vLLM chooses a random one per request, and the resumed part would be a fresh sample.

Recovery is restart. With one replica, recovery was the engine's restart: 108–185 s on vLLM and 130–138 s on SGLang (median per run, default mode). The gap between the engine answering its health check and the first recovered position was under 0.5 s (median per run); it is known only to about one second, because the harness and the gateway poll the engine's health independently, so a few individual gaps come out slightly negative. The jump in vLLM's restart time between runs came from weight loading (45–47 s in the first 44 starts, 69–126 s in 33 of the next 35), not from the gateway; we did not establish why.

Failover. With a standby replica on a second machine, recovery took 0.05–0.08 s (median per run; Table 8): detecting the broken stream, checking the other replica's health, and prefilling the prompt and the delivered tokens there. That is more than three orders of magnitude faster than restart, with the same exactly-once delivery. (Measured at the client, version 2 reported 0.16–0.17 s; the difference is the client delay described above. The first trial of a run was occasionally slower, up to about 0.4 s.)

Exact text needed matching compiled state in our runs. In batch-invariant mode each machine was repeatable on its own, but the second machine's failure-free answer differed from the first's from character 554 on, a single word after which the two continue differently, and the recovered text matched in 0/20 trials. The difference was not the mode setting, which the replica's process environment confirmed, nor the hardware or driver (same instance type, image and driver), nor the Python packages: before the second batch-invariant run we set the 21 packages that differed to the first machine's versions, so that their `pip freeze` lists were byte-identical, and the answer still split at the same character. What remained different was each machine's compiled state: the torch.compile cache vLLM keeps (235 MB on the first machine, 115 MB on the second), FlashInfer's cache (792 kB against 12 kB), and TorchInductor's. With the first machine's three caches

copied to the second, the two machines' failure-free answers were identical and the recovered text matched in 10/10 trials. We did not isolate which cache matters, or why artifacts compiled on identical machines differ; a plausible candidate is kernel autotuning, which chooses among variants by measured speed, but we did not test it. The finding is therefore limited to what we observed: in the evaluated configuration, synchronizing the compiled-kernel caches restored output equivalence between the replicas, and the responsible artifact remains unidentified. For failover it means that a standby gave exactly-once delivery and sub-second recovery regardless, but the same text as a failure-free run only when it ran in a deterministic mode and started from the first machine's compiled state. In default mode, failover with the second machine's own caches gave the failure-free text in 2 of 10 trials at temperature 0, where the references differed as after restart, and in 10 of 10 at temperature 0.8; a sampled token is less sensitive to a last-digit difference than a greedy choice at a near-tie, but ten trials do not show how much less.

Overhead. We expected, before these runs, that exactly-once delivery would cost little per token, since the log grows by one entry per step. That held (Table 9): about +0.2 ms per output token on vLLM (0.5%), whose stream through the gateway also carries token ids, no measurable difference on SGLang, and end-to-end time within 0.06 s on both. Before the first token the gateway added about 5 ms on vLLM (9 ms with `fsync`), where it makes extra round trips to the engine (two health checks and `/tokenize`), and under 1 ms on SGLang. Version 2 reported 32–45 ms here, and could not explain why that did not show end to end on SGLang: it was the measuring client's delay of about one token, not the gateway. The log's timestamp precedes the write, the optional `fsync` and a local socket send, so these figures can be lower than a client would see by about a millisecond, plus the `fsync`.

10 The fixes under concurrent load

Every experiment so far issued one request at a time. Two configurations removed the contradictions there: each engine's deterministic mode and, on vLLM only, disabling automatic prefix caching. We measured what each costs under concurrent load, and whether it still makes temperature-0 output unique when requests share batches.

Setup. One `g5.xlarge`, the same model and engine versions, 4,096-token context cap, commit `b164136`. `harness/run_throughput.py` starts the server, sends four unrecorded warm-up requests, then, for each concurrency $C \in \{1, 4, 16, 32\}$, sends $\max(12, 4C)$ requests with C in flight. Requests cycle through the six prompts of Section 7 at temperature 0 with `ignore_eos`, so every request generates exactly 256 tokens. Each configuration ran twice, interleaved; the two repetitions agree within 1 token/s at every level. That is 10 runs and 2200 requests, none of which failed (Table 10).

The deterministic modes cost half the throughput and kept output unique. Both modes ran at 0.46–0.57× their engine's default throughput, with time per output token roughly doubled, and the relative cost shrank slightly with load. Both gave exactly one output per prompt in every run, across about 37 requests per prompt at up to 32 in flight. The property that made recovery exact in Sections 7 and 9 therefore holds under batching; recovery itself was not injected under load.

Disabling the prefix cache did not survive batching. Without prefix caching vLLM lost no throughput on this workload, but under load it gave 4–10 distinct outputs per prompt, as many as with the cache on, against at most two in the sequential runs. The default modes behaved alike: 3–9 distinct outputs per prompt, consistent with output depending on batch composition [6]. The sequential finding of Section 6, that disabling vLLM's prefix cache removes the contradictions, is therefore not a fix for a server under load. The prompts here are about 20 tokens, so the cache had almost nothing to save; time to first token at C=32 still rose from 112 to 191–221 ms without it, and workloads with long shared prefixes would pay more. That was not tested.

11 Recompute versus a KV copy

The gateway recovers by recompute: the replica that takes over rebuilds the KV cache by prefilling the prompt and the delivered tokens. The proposal behind this project hypothesised a crossover: recompute wins for short contexts, and restoring a saved copy of the KV cache wins beyond some length. We measured both sides.

Recompute. `harness/run_prefill.py` sends prompts of L pseudo-random token ids with `max_tokens`=1 and records the time to the first token, which is the prefill a recompute recovery pays. Every prompt is distinct, so nothing is served from a cache; five requests per length, one at a time. Runs with prefix caching on and off agree within 4 ms at every length, and the spread over five requests is at most 3 ms. Prefill took 0.14–12.5 s from 512 to 32,768 tokens, and its rate fell above 2K tokens as attention's

quadratic cost took over. At 32K tokens the GPU is at its memory limit; one sweep needed vLLM's `--gpu-memory-utilization 0.94` to start, which changes the memory reserved for the cache, not prefill speed.

Copy. `harness/kv_copy_bench.py` allocates a buffer of the model's KV size for L tokens (2×32 layers $\times$ 8 KV heads $\times$ 128 $\times$ 2 bytes = 128 KiB per token) and times GPU to pinned host memory and back, and host to the local disk with `fsync` and back, with the page cache dropped before every read. `harness/kv_net_bench.py` moves the same buffers from one machine's GPU over TCP into the other's GPU (same availability zone, no placement group), with one and with four TCP streams. Each network session ran a size sweep, 15 back-to-back transfers of the 4.3 GB buffer, and the sweep again, about 150 GB in all, so that an exhausted burst allowance would show as a falling rate. These are raw copies on an otherwise idle GPU: lower bounds on the cost of a checkpoint, not a checkpoint implementation.

There is no crossover where it matters. Table 11 gives the answer by restore path. From host memory (about 13.4 GB/s each way) a restore was 25–39$\times$ faster than recompute at every length. From the second machine it was 1.9–3.1$\times$ faster with four streams and 1.05–1.66$\times$ with one. The network rate was flat, 5.0 Gbit/s with one stream and 9.9 Gbit/s with four, in every sustained transfer and both sweeps; 5 Gbit/s is the single-flow limit EC2 documents outside a cluster placement group, so a checkpoint stream over one connection gives up half the benefit. Only the local disk, read at about 350 MB/s, shows the hypothesised crossover, at 32K tokens, the top of the range. The margin grows with context because prefill grows faster than linearly and the copy linearly: at 512 tokens a copy from the standby saves 0.07 s, at 32K about 8.4 s. A host-memory copy survives only the engine process failing, not the machine; for failover, the second-machine columns are the relevant ones.

What this leaves open. A restore needs the KV cache to be on the standby when the failure happens, so a real checkpoint has to stream it while the request decodes. One request adds 128 KiB per output token, about 4 MB/s at 33 ms per token, or about 115 MB/s for 32 concurrent requests, under 10% of the four-stream link; but whether streaming at that rate slows decoding cannot be measured with raw copies on an idle GPU.

12 Checkpoint policy: a simulation

This section reports a simulation, not a measurement. The original plan for the last research question was an adaptive policy that decides per request whether to checkpoint, recomputing short requests and checkpointing long ones. Section 11 removed the premise: restoring from the standby wins at every length. What remained open was how often to stream new KV entries, given an unknown cost on the decode path. Before building a live system we simulated the question with a model whose hardware costs are read from the committed logs of the measurements above, and wrote down, before running it, the rule that would decide whether the live system was worth building (`docs/phase5-plan.md`).

Model. Inputs read from logs: the recompute cost $R(n)$ and copy rates of Section 11, the four-stream network rate, 33.1 ms per output token from Section 10, and 0.038 s of failover detection and switching from Section 9 (the failover recovery there, minus the prefill it includes; version 3 reran the simulation with this input taken from the gateway's log instead of the client, which lowered it from 0.125 s and lowered every simulated recovery time by about 0.09 s, without changing any decision). Assumptions, swept or stated: the share $\beta \in \{0, 0.1, 0.5, 1\}$ of a checkpoint's copy time that stalls decoding; a mean time between replica failures of 1 day, 1 hour or 10 minutes, fail-stop and Poisson, with a first-order expected cost (at most one failure per request; the largest failure rate times request duration in the grid is 0.13); three workloads of 2,000 requests each, *chat* (prompts of 64–2,048 tokens, outputs of 64–512), *long* (prompts of 4,096–28,000 tokens, outputs of 256–2,048) and *mixed* (80% chat, 20% long); one request at a time; a standby always available. Policies decide at the end of prefill and every 16 output tokens: *recompute* (the gateway as built), *prompt-only* (checkpoint the prompt's KV once), *fixed-k* (after prefill and every k tokens, $k \in \{16, 64, 256, 1024\}$), and two adaptive rules, the proposal's inequality and one weighing the remaining time of the request against the saving per failure.

Decision rule. For each configuration the baseline is the best non-adaptive policy by expected completion time, chosen in hindsight. An adaptive policy wins if it lowers expected completion time by at least 5%, or 99th-percentile recovery latency by at least 5% while being at most 0.1% slower in expectation. A live adaptive policy would be built only if one won on the mixed workload for $\beta \in \{0.1, 0.5\}$ at a failure rate of one hour or rarer.

Result: no adaptive policy. Neither adaptive rule won a single configuration, in any workload. At a failure every hour, a 15.5 s mixed request fails with probability of about 0.4%, so even removing all recovery time saves at most a few milliseconds per request on average, while a checkpoint that stalls decoding for a tenth of its copy time already costs 0.21–0.25% in this workload. Both adaptive rules work this out and stop checkpointing as soon as checkpoints cost anything ($\beta \geq 0.1$ at failure rates of an hour or rarer), so they match the best non-adaptive policy, which is then recompute, and cannot beat it (Table 12). Adaptivity buys nothing measurable in expected completion time, for the modeled workloads, policies and failure rates; we do not claim it for others.

What checkpointing buys is tail latency. For failures during decoding, every checkpointing policy cuts recovery sharply: p99 from 9.00 s under recompute to 0.97 s with prompt-only and 0.32 s with a checkpoint every 16 tokens. Prompt-only captures most of the gain for one copy per request. A failure during prefill (3.8% of failure positions in the mixed workload) is still recovered by redoing the prefill under every policy. The simulation thus narrowed the live work to measuring the decode-path cost of a fixed policy and the recovery it buys.

13 A KV checkpoint inside vLLM

vLLM 0.31 can offload KV-cache blocks from the GPU to other storage through its `OffloadingConnector`. We used it unmodified as a checkpoint: both replicas write blocks to, and look blocks up in, one directory that the standby can read, and the gateway resumes a failed request on the standby as before. If the blocks for the prompt and the delivered tokens are there, the standby loads them instead of recomputing them.

Setup. Two `g5.xlarge` machines in one availability zone, vLLM 0.31.0, Llama 3.1 8B Instruct, a 17,000-token context cap, greedy decoding, 256 output tokens. The first machine runs the gateway and replica 0; the second runs the standby and exports a 6 GB RAM disk over NFS (`nconnect=4`), mounted on both at the same path. Each replica's connector keeps a 4 GiB CPU tier backed by a second tier in that directory; block hashes use the same seed in both processes, so a block written by one is found by the other. Three modes: *none*, no connector, so the standby recomputes as in Section 9; *prompt*, offloading the prompt's blocks only (vLLM's default); and *all*, every full 16-token block, prompt and output. Prompts are pseudo-random token ids, distinct in every request; the shared directory is emptied before every failover trial, and vLLM's leftover shared-memory file before every engine start.

Cost while serving. Replica 0 alone served requests with 4,096-token prompts, one and four at a time, two repetitions per mode in interleaved order (commit `d9dd746`; Table 13). Time per output token was unchanged within 0.1% at one request at a time, and throughput was within 0.6% at four. In the simulation's terms, the decode stall β was close to 0, below the whole range simulated, because vLLM copies blocks off the GPU asynchronously. The cost moved to the first token: with *prompt* it arrived about 55 ms (5%) later, in both repetitions; with *all* the two repetitions differed (+36 and +3 ms), for reasons we did not establish. The 6 GB RAM disk filled during the C=4 batches (each prompt is 537 MB of KV), after which new blocks could not be written to the shared tier, so the C=4 figures cover only part of the copying.

Two sets of runs. The failover runs were done twice. The first set, a week before the second, used the gateway as evaluated in Section 9, with one fix (the event encoding, below); it showed that the text a client concatenates could repeat earlier text, which the gateway then learned to remove. The second set (commit `1b978f2`, Tables 14 and 15) repeated the single-request trials with that fix, added a check of the client's text, and added failures with 4, 16 and 32 requests in flight; its first two attempts were discarded, one because the gateway lost requests (below) and one because two copies of the experiment ran on the same machines at once. Recovery figures of the first set, taken from the gateway's log, agree with the second within 0.05 s.

Recovery, one request at a time. Replica 0 was SIGKILLed once 128 output positions had reached the client, with prompts of 4,096 and 16,384 tokens (Table 14). All 30 trials delivered every position exactly once, and in all 30 the client's text was right. With a checkpoint the standby resumed in 0.64–0.65 s instead of 5.14 s at 16K tokens, and in 0.24 s instead of 1.08 s at 4K. The standby demonstrably used the checkpoint: its loaded bytes rose by 537 MB at 4K and 2,147 MB at 16K with *prompt*, exactly the prompt's KV at 128 KiB per token, and slightly more with *all*, which also stores the decoded blocks; in the first set, its log reported an external prefix-cache hit rate of 98.8–99.8%. *All* was no faster than *prompt*, since only 128 decoded tokens were at stake. We did not break the remaining 0.2–0.6 s down; it includes loading up to 2 GB over NFS and recomputing what was not stored. In both checkpoint modes, and in both sets of runs, the first 4K trial took 0.56–0.60 s, against 0.20–0.24 s for almost all the others,

consistent with a first-use cost on the freshly started standby (the first set also had one later slow trial, and a slow first trial without a checkpoint).

Recovery with many requests in flight. With C requests running when replica 0 died, every request was resumed on the standby (Table 15): all 360 requests received every position exactly once. Without a checkpoint the standby must recompute all C requests at once, about 37,000 tokens of prompt and delivered output at C=32, so requests queue: median recovery rose from 0.98 s at C=4 to 2.73 s at 16 and 4.86 s at 32, with the slowest request at 15.2 s. With *prompt* the standby loaded the prompts instead (537 MB at C=4, about 4.8 GB at 32), and median recovery was 0.27, 0.95 and 1.48 s, the slowest 8.0 s; with *all* at C=16 it was 0.53 s. The checkpoint's benefit therefore grows with the number of requests a failure interrupts, not only with their length. Time to first token in this table includes queueing on replica 0, which served all C requests at once.

The checkpoint is paid for on every request. Time to first token on replica 0 rose from 1.40 to 1.51 s at 4K (8%) and from 5.41 to 5.70 s at 16K (5%), consistent with the 5% of the serving-cost runs. At 16K a failure saves about 4.5 s and every request pays about 0.3 s, so in expected completion time the checkpoint pays off only if more than about one request in 15 meets a failure. The measured cost thus lands on the prefill, which the simulation did not model, rather than on decoding, which it did. The live measurements therefore support the simulation's qualitative conclusion, that checkpointing buys tail recovery latency rather than average speed, but they do not validate its quantitative predictions: its overhead column assumed a decode stall, and the first-token cost actually measured is outside its model.

Positions were exactly-once; the text, at first, was not. In the first set of runs, in 9 of 30 trials, the first entry after the resume carried as its text 24,000–97,000 characters: the whole decoded prompt, then the text already delivered, then the new token's text. The token ids were correct and every position arrived once, but a client that concatenates the text, as streaming clients do, would have displayed the prompt and repeated what it had already shown: duplication at the level a user sees. The 9 cases were exactly the trials in which the last delivered token streamed with empty text, as a token holding only part of a UTF-8 character does (the random-token prompts make the model emit such tokens), and none of the other 21; so a resume whose token-id prompt ends inside a character made vLLM's streamed text start from the beginning of the sequence. We did not trace this inside vLLM. None of the 190 trials of Section 9, with natural-language text, showed it, but a kill inside a multi-byte character (an emoji, CJK text) could. The gateway now checks the first entries after a resume against the decoding of the token ids (`/detokenize` of prompt plus delivered plus new ids, minus that of prompt plus delivered ids), holds back a token whose decoding ends in an incomplete character, removes text the engine repeated, and otherwise keeps the engine's text, which omits special tokens that `/detokenize` shows. In the second set the engine repeated earlier text after a resume 64 times in 390 requests, and the gateway removed it every time. The client's text was right in 387 of 390 requests (`analysis/recheck_text.py`, with the model's tokenizer). The 3 exceptions, at C=32, involved no failure: each was a request whose token-id prompt ended inside a character that the first output token completed. vLLM repeated the prompt as that token's text, and the gateway, which then decoded the output ids without the prompt, did not recognise the repetition and passed it on. Decoding with the prompt as context fixes this (commit `852962e`, with a test against a mock engine that imitates it); that fix has not been run on a GPU. A text prompt, which the gateway tokenizes itself, ends on a whole character and cannot trigger the case.

Problems found by the concurrent runs. Three faults in our own code surfaced only with many requests in flight; all are fixed. The gateway's HTTP server kept Python's default backlog of five pending connections, so when 32 clients connected at once, three connections were reset and those requests never started; the backlog is now 1,024, and the client chooses the request id, so that a submission whose connection failed is resumed if the gateway started it and submitted again if not. The client delay described in Section 9 was found while checking the kill timing of these runs. And the reader that replaced `iter_lines` did not translate the error raised when a killed engine's chunked stream is cut, so requests ended at the kill instead of failing over; this was caught in the first trials and fixed before the runs reported here.

An event-stream bug. In the first set of runs, one interrupted run had a repeated text containing the character U+0085. The gateway sent its event stream without a charset, so the Python `requests` library decoded it as Latin-1, and splitting the decoded text into lines treated U+0085 as a line break, cutting one event in two; the client's checker reported a missing position. The gateway now escapes every non-ASCII character in its JSON events, declares UTF-8, and its client splits lines as bytes (commit `c1b065e`). The 190 trials of Section 9 ran on the earlier code; none of them hit any of the problems in this section.

14 Discussion

Two engines, two mechanisms, one consequence. vLLM's variation was deterministic and tied to its prefix cache; SGLang's was tied to server start and was not repeatable across starts. A client cannot tell these apart, and for a client they have the same effect: the server that answers a retry is freshly started, and a freshly started server did not reproduce the text the client had been shown, in 25 of 30 vLLM and 24 of 30 SGLang trials.

Retry is not replay. The usual argument for retrying an LLM request is that at temperature 0 it is idempotent up to duplication. On the default vLLM configuration measured here, it was not: the server that answers a retry is a fresh process, and a fresh process gave a different greedy answer from the warm one that had been streaming. For a chat client this means a visible rewrite of text already shown in most trials on both engines; for an agent that acted on a partial tool call or plan, it means the action was based on text that the final answer contradicts.

What a client and an operator should do. With default engine settings, recompute is the only client-side strategy that never contradicted what was shown, at the price of a final answer that no failure-free run produced, and a few tens of milliseconds of extra prefill. Naive retry with deduplication is unsafe unless the client verifies that the replay begins with D, and should replace rather than splice when it does not. On vLLM, disabling automatic prefix caching removed the contradictions for sequential requests, but under concurrent load it no longer made the output unique (Section 10), so it is not a fix for a busy server. On SGLang, disabling the radix cache did not help even sequentially. The combination that worked on both engines was an engine deterministic mode on the server and recompute on the client: at temperature 0 it reproduced the failure-free output exactly in all 35 trials (Section 7), without any change to the engines' code. The client ended with the same text as a failure-free run, the outcome exactly-once delivery would guarantee. That is equivalence in the tested configurations, not an exactly-once protocol: it relied on greedy decoding and the operator enabling the mode, and it costs about half the throughput (Section 10). Under load the modes still gave one output per prompt, but recovery was not injected under load.

What the gateway changes. Client-side recovery could not remove contradictions without the operator enabling a deterministic mode, and left every client to implement recompute and deduplication correctly. The gateway makes exactly-once delivery a property of the serving layer: every position once, in order, for unmodified clients and engines, in every trial we ran, including with 32 requests in flight, at a per-token cost below 1%. What it cannot add is determinism. Whether the recovered text equals a failure-free run is still decided by the engine: its deterministic mode, its sampler, and, after failover, whether the standby matches the failed replica's compiled state, which in our runs it did only when the caches were copied. Version 1 of this paper argued that exactly-once recovery above temperature 0 would also need the sampler's state. On vLLM 0.31 with a fixed request seed it needs no engine change, because the sampler derives each draw from the seed and the position; on SGLang, whose default mode ignored the seed, it was not available.

What a checkpoint adds. Recovery time fell in three steps across this study: from minutes, when a crashed engine had to restart, to 0.05–0.08 s with a standby for a short prompt, and, with a KV checkpoint on the standby, to under a second for long prompts as well (0.64 s instead of 5.14 s at 16K tokens), and to 1.48 s instead of 4.86 s (median) when 32 requests fail together. The checkpoint needed no engine change, only vLLM's offloading pointed at shared storage, and the gateway delivered every position exactly once throughout, although the text needed the fixes of Section 13. Its price was not where we expected it: decoding was not slowed, but every request's first token came 5–8% later. Measured and simulated results agree on the economics: at plausible failure rates no checkpoint pays for itself in average completion time, and a per-request adaptive policy has nothing to gain. The case for a checkpoint is a target for worst-case recovery, for long contexts and for agents that wait on a stream; prompt-only checkpointing captures most of that gain (Figure 2).

15 Related work

Fault-tolerant LLM serving. DéjàVu [12] streams and replicates KV cache to recover from failures and reports server-side recovery time; it does not characterize what streaming clients receive. SpotServe [9] migrates in-flight context when preemptible instances are reclaimed with warning, and Llumnix [13] live-migrates requests for load balancing. Both handle planned moves; this study targets unplanned fail-stop crashes. **Serving engines.** Orca [15] introduced iteration-level scheduling, vLLM [7] block-managed KV memory, and SGLang [16] prefix sharing via RadixAttention. **Delivery semantics.** Chandy and Lamport [3] define consistent snapshots, Elnozahy et al. [5] output commit, and Flink [2] and Spark Structured Streaming [1] build exactly-once sinks on them. **Inference nondeterminism.** He et al. [6] show that LLM inference at tempera-

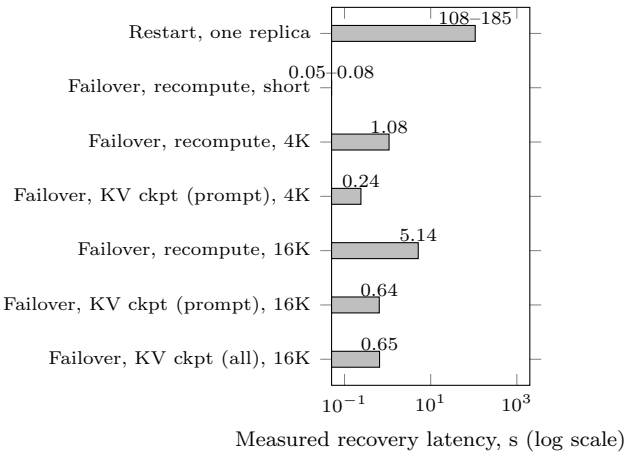


Figure 2: Measured recovery latency (kill to the gateway's commit of the next position) by mechanism, vLLM, one request at a time; no simulated values. Restart and the short-prompt failover are medians per run from Section 9 (bar at the lowest run median); 4K and 16K are medians over five trials from Section 13. Serving cost: none for restart and recompute failover; for the KV checkpoint, under 1% in decoding and 5–8% in time to first token.

ture 0 is not reproducible across server load because kernels are not batch-invariant, and build batch-invariant kernels. Our observation is the fault-tolerance consequence: requests here were issued one at a time, yet output still depended on what the server had served before, so a retry after a crash is not a replay. **KV-cache storage and transfer.** CacheGen [8] compresses and streams KV caches over the network to avoid recomputing long contexts, and Mooncake [10] builds serving around a disaggregated KV-cache store; both target efficiency in failure-free serving, which is the machinery our checkpoint borrows for recovery. **Checkpoint intervals.** Classic results on optimal checkpoint intervals [14, 4] balance checkpoint cost against expected lost work; our simulation applies the same reasoning per request and finds that, at plausible failure rates, the balance favours checkpointing only for tail latency.

16 Threats to validity

- **Trials are not independent samples.** In vLLM, output was deterministic given server history, so 5 trials per configuration are repeated observations of one or two histories, and counts such as "25 of 30" describe how many configurations and histories were affected, not a sampled rate. In SGLang the cold output varied from start to start, so its trials do sample that variation, but 5 per configuration is too few to estimate rates.
- **Six prompts, one model, one GPU type.** The main grid used one prompt; five more were tested only at temperature 0 and the 128-chunk kill point, and vLLM's batch invariance only on the main prompt. Divergence positions are prompt-specific (two of five extra prompts showed no vLLM divergence within 256 tokens). Other models and tensor-parallel serving may differ.
- **Default configurations, different kernels.** Each engine ran with its own defaults on this GPU (vLLM with PyTorch sampling because FlashInfer's sampler could not compile on the host image; SGLang with FlashInfer attention and sampling inside its container). The comparison is between default behaviours, not kernels.
- **SGLang seed.** The harness's OpenAI-style seed had no observable effect on SGLang, so its temperature-0.8 results are for unseeded sampling and are not comparable to vLLM's seeded runs. Its temperature-0 results do not involve sampling.
- **vLLM sampler substitution.** vLLM ran with PyTorch sampling instead of FlashInfer's sampler, because of the missing CUDA toolkit. The temperature-0 results do not involve random sampling, but the 0.8 results may differ under FlashInfer.
- **Mechanism.** For vLLM, the ablation shows that the dependence on server history requires automatic prefix caching, and re-tokenization was ruled out for recompute. For SGLang, neither the radix cache nor the server seed was required, and the source of the start-to-start variation was not identified. That both engines' deterministic modes removed every contradiction and made recompute exact at temperature 0 supports, but does not prove, a numerical explanation [6]; we did not identify which kernel differs. The ablations and follow-up runs covered only the 128-chunk kill point.
- **Re-tokenization in recompute.** Recompute sends D as text, which the engine re-tokenizes. We checked this directly for vLLM (Section 7): re-tokenization matched in all 10 trials checked, at the 128-chunk kill on the main prompt. Other prompts or kill points could tokenize differently at the boundary.
- **Concurrent failures, narrowly.** Failures with requests in flight were injected only in Section 13: one prompt length (1,024 random token ids), 4, 16 and 32 requests, three trials per cell, the kill timed on one request, default mode only. Exact text under concurrent failure in a deterministic mode was not tested.

- **Times through the gateway.** These come from the gateway's log (Section 9), stamped when an entry is appended, before it is written, optionally fsynced and sent; a client on the same machine sees it about a millisecond later, plus the `fsync`. Version 2's client-side figures were too long, by up to about one inter-token interval, or about 0.1 s after a failover.

- **Fail-stop only; latency includes restart.** `SIGKILL` is a clean crash; GPU hangs and silent corruption are not modeled. With a single instance, recovery includes model reload; a warm standby would be faster. Restart time also drifted during the grid (see H3).

- **Gateway evaluation scope.** One prompt, one model, requests one at a time, 10 trials per run; failover only on vLLM and only with the kill after 128 positions; one pair of machines. The compiled-cache finding rests on three batch-invariant runs on that pair, and which cache matters was not isolated. The gateway's own durability was tested only by killing its process with the log on local disk; a lost disk, a replicated gateway, and a gateway restart under concurrent requests were not tested. Recovery times below one second are bounded by health-check polling. Its overhead is that of this implementation.

- **Generalizability.** Every measurement used one model (Llama 3.1 8B Instruct), one GPU type (A10G, one per machine), one cloud and machine image, two engine versions, and, for the gateway and checkpoints, one prompt or one synthetic prompt shape. Larger models with tensor or pipeline parallelism have more state to lose, longer restarts and different kernels; other GPUs differ in memory and copy bandwidth, which moves every recompute-versus-copy ratio; and engines change quickly, so the specific behaviours found here (the prefix-cache dependence, the start-dependent output, the echoed text) may be fixed or replaced in later versions. The delivery problem and the gateway's protocol do not depend on these details; the numbers do.

- **Checkpoint scope.** Section 11 measured raw copies of contiguous buffers on an idle GPU. Section 13 used one prompt shape (random token ids), one kill position, five trials per cell and one pair of machines; its shared store lives in the standby's memory and is lost with the standby, it filled up at four concurrent requests, and a failure during prefill still forces a full recompute. A SIGKILL is detected at once; a hung GPU would add a detection timeout to every mode. The gateway's text check recognises special tokens by their text (`<|...|>`), not their id, so a model that writes such a string as ordinary text inside the first entry after a resume could lose it; and its fix for prompts that end inside a character was tested against a mock engine only. The simulation of Section 12 is a first-order model with stated assumptions, and its conclusions are only as good as those assumptions.

17 Conclusion and future work

In the default configurations of both vLLM and SGLang, the failure-free output for a fixed prompt was not unique at temperature 0, so the restarted server that answers a retry did not reproduce what the client had been shown: naive retry contradicted displayed text in most trials on both engines, and on three (vLLM) and four (SGLang) of five further prompts. The cause was not re-tokenization; in vLLM it required prefix caching, and in SGLang it was tied to server start. Each engine's deterministic mode with recovery by recompute reproduced the failure-free output exactly in every temperature-0 trial, and under concurrent load these modes kept the output unique at about half the throughput; disabling vLLM's prefix cache did not.

A gateway that numbers and logs output positions and resumes from the logged token ids delivered every position exactly once in all 190 injected failures on both engines, for unmodified clients that keep their position, at a per-token cost below 1%, and in all 360 requests of failures with up to 32 requests in flight; with a standby replica on a second machine it recovered in under 0.1 s instead of minutes. After a resume, vLLM sometimes repeated earlier text; the gateway removes it. Exact text after recovery remained the engine's property: it held in deterministic mode after restart, and after failover only once the failed replica's compiled-kernel caches had been copied to the standby; which artifact matters is not identified.

Restoring a KV copy from the standby was faster than recomputing it at every length measured, so the question became how often to checkpoint. A simulation driven by the measurements, judged by a rule fixed in advance, found no value in an adaptive per-request policy. Measured live, vLLM's own KV offloading to storage the standby can read cut recovery at 16K tokens by a factor of eight, and by a factor of three when 32 requests failed together, with every position delivered exactly once, no measurable decoding cost, and 5–8% later first tokens: a tool for bounding worst-case recovery, not for average speed.

Next stages: (1) isolate which compiled artifact makes identical machines differ, and whether autotuning is the cause; (2) concurrent failures in deterministic mode, with natural-language prompts and a replicated

gateway; (3) checkpoint a long prefill in chunks as it is computed, and keep the store somewhere that survives the standby; (4) other models, GPUs and tensor-parallel serving.

Artifact. Code, raw logs, and the table-generating script are at `https://github.com/rmedipallycic/llm-inference-fault-tolerance` (Apache-2.0). Every number above traces to a committed log file; the simulation's inputs are read from those files, and its output is kept apart from the measurements.

Acknowledgments

The harness code, analysis scripts, and much of the text of this paper were developed with substantial help from an AI assistant (Claude, Anthropic). The author directed the work, ran all experiments on AWS, and is responsible for every reported result; each number is generated by `analysis/make_tables.py` from the raw logs committed to the repository.

Table 3: Client-visible outcome per configuration (5 trials each), generated from the raw logs by `analysis/make_tables.py`. $|D|$: characters delivered before the kill (two values, or a range, when trials differed). Contradicts D: the naive replay disagreed with text already shown. $F{=}G$, $F{=}G'$: the final text equals the first or second failure-free reference.

Config	Strategy	T	Kill (chunks)	$\lvert D\rvert$ (chars)	Contradicts D	$F{=}G$	$F{=}G'$
vLLM	naive	0	25	113	0/5	5/5	0/5
vLLM	naive	0	128	643	5/5	5/5	0/5
vLLM	naive	0	230	1145	5/5	5/5	0/5
vLLM	naive	0.8	25	105	5/5	5/5	0/5
vLLM	naive	0.8	128	590	5/5	5/5	0/5
vLLM	naive	0.8	230	1079	5/5	5/5	0/5
vLLM	recomp.	0	25	113	n/a	0/5	0/5
vLLM	recomp.	0	128	625/643	n/a	0/5	0/5
vLLM	recomp.	0	230	1143/1145	n/a	0/5	1/5
vLLM	recomp.	0.8	25	105/114	n/a	4/5	0/5
vLLM	recomp.	0.8	128	556/590	n/a	0/5	0/5
vLLM	recomp.	0.8	230	1079/1085	n/a	0/5	1/5
vLLM	recomp. (ids)	0	128	625/626	n/a	0/5	1/5
vLLM, no APC	naive	0	128	612	0/5	5/5	5/5
vLLM, no APC	naive	0.8	128	601	0/5	5/5	5/5
vLLM, no APC	recomp.	0	128	612	n/a	0/5	0/5
vLLM, no APC	recomp.	0.8	128	601	n/a	5/5	5/5
vLLM, no APC	recomp. (ids)	0	128	612	n/a	0/5	0/5
vLLM, batch-inv.	naive	0	128	612	0/5	5/5	5/5
vLLM, batch-inv.	naive	0.8	128	556	0/5	5/5	5/5
vLLM, batch-inv.	recomp.	0	128	612	n/a	5/5	5/5
vLLM, batch-inv.	recomp.	0.8	128	556	n/a	5/5	5/5
SGLang	naive	0	25	113	0/5	1/5	0/5
SGLang	naive	0	128	643	4/5	1/5	0/5
SGLang	naive	0	230	1144	5/5	0/5	0/5
SGLang	naive	0.8	25	76–121	5/5	0/5	0/5
SGLang	naive	0.8	128	282–662	5/5	0/5	0/5
SGLang	naive	0.8	230	1120–1173	5/5	0/5	0/5
SGLang	recomp.	0	25	113	n/a	0/5	0/5
SGLang	recomp.	0	128	623/643	n/a	0/5	0/5
SGLang	recomp.	0	230	1113–1163	n/a	3/5	1/5
SGLang	recomp.	0.8	25	100–136	n/a	0/5	0/5
SGLang	recomp.	0.8	128	608–686	n/a	0/5	0/5
SGLang	recomp.	0.8	230	1115–1206	n/a	0/5	0/5
SGLang, no radix	naive	0	128	643	5/5	1/5	0/5
SGLang, no radix	naive	0.8	128	599–682	5/5	0/5	0/5
SGLang, no radix	recomp.	0	128	643	n/a	0/5	2/5
SGLang, no radix	recomp.	0.8	128	430–683	n/a	0/5	0/5
SGLang, fixed seed	naive	0	128	643	5/5	1/5	0/5
SGLang, fixed seed	naive	0.8	128	456–691	5/5	0/5	0/5
SGLang, fixed seed	recomp.	0	128	623/625	n/a	0/5	0/5
SGLang, fixed seed	recomp.	0.8	128	588–670	n/a	0/5	0/5
SGLang, determ.	naive	0	128	625	0/5	5/5	5/5
SGLang, determ.	naive	0.8	128	666	5/5	5/5	0/5
SGLang, determ.	recomp.	0	128	625	n/a	5/5	5/5
SGLang, determ.	recomp.	0.8	128	666	n/a	0/5	0/5

Table 4: Latency in seconds, median (min–max). Restart: kill to server ready. TTFT: server ready to first recovered chunk. Recovery: kill to first recovered chunk.

Config	Strategy	Trials	Restart	TTFT after restart	Recovery
vLLM	naive	30	116.6 (108.1–122.1)	0.26 (0.23–0.28)	116.9 (108.3–122.4)
vLLM	recomp.	30	177.7 (110.1–195.2)	0.27 (0.23–0.32)	178.0 (110.4–195.5)
vLLM	recomp. (ids)	5	116.1 (107.1–119.1)	0.59 (0.58–0.60)	116.7 (107.7–119.7)
vLLM, no APC	naive	10	109.6 (107.1–120.1)	0.25 (0.23–0.28)	109.9 (107.3–120.4)
vLLM, no APC	recomp.	10	111.1 (108.1–120.1)	0.28 (0.25–0.30)	111.4 (108.4–120.4)
vLLM, no APC	recomp. (ids)	5	116.1 (108.1–119.1)	0.61 (0.60–0.62)	116.7 (108.7–119.8)
vLLM, batch-inv.	naive	10	122.6 (119.1–131.1)	0.27 (0.26–0.28)	122.9 (119.4–131.4)
vLLM, batch-inv.	recomp.	10	128.1 (118.1–132.1)	0.29 (0.28–0.30)	128.4 (118.4–132.4)
SGLang	naive	30	133.2 (125.1–145.2)	0.40 (0.33–0.49)	133.6 (125.6–145.5)
SGLang	recomp.	30	129.2 (123.1–193.2)	0.42 (0.35–0.51)	129.6 (123.5–193.7)
SGLang, no radix	naive	10	184.7 (178.2–188.2)	0.40 (0.35–0.42)	185.1 (178.6–188.6)
SGLang, no radix	recomp.	10	185.7 (172.2–195.2)	0.43 (0.40–0.48)	186.1 (172.6–195.7)
SGLang, fixed seed	naive	10	132.7 (129.1–142.2)	0.41 (0.35–0.47)	133.1 (129.6–142.6)
SGLang, fixed seed	recomp.	10	131.6 (127.1–135.1)	0.44 (0.37–0.46)	132.1 (127.5–135.6)
SGLang, determ.	naive	10	133.6 (127.2–146.2)	1.72 (0.34–4.02)	135.4 (130.5–146.5)
SGLang, determ.	recomp.	10	134.6 (127.2–142.2)	1.70 (0.38–4.07)	135.1 (130.6–142.6)

Table 5: Recompute (ids), vLLM, temperature 0, kill at 128 chunks. Re-tok.: re-tokenizing D gave the generated token ids.

Config	T	n	Re-tok.	$F{=}G$	$F{=}G'$
vLLM	0	5	5/5	0/5	1/5
vLLM, no APC	0	5	5/5	0/5	0/5

Table 6: Five further prompts, temperature 0, kill at 128 chunks, 5 trials per strategy. Contra.: naive replay contradicted D. Exact: recompute's final text equals G or G'.

Config	Prompt	$G{=}G'$	Contra.	Exact
vLLM	coffee	0/2	5/5	0/5
vLLM	crisis	2/2	0/5	0/5
vLLM	story	0/2	5/5	0/5
vLLM	tcp	2/2	0/5	0/5
vLLM	threads	0/2	5/5	0/5
vLLM, no APC	coffee	2/2	0/5	5/5
vLLM, no APC	crisis	2/2	0/5	0/5
vLLM, no APC	story	2/2	0/5	0/5
vLLM, no APC	tcp	2/2	0/5	0/5
vLLM, no APC	threads	2/2	0/5	0/5
SGLang	coffee	0/2	3/5	0/5
SGLang	crisis	0/2	4/5	0/5
SGLang	story	0/2	5/5	0/5
SGLang	tcp	1/2	0/5	2/5
SGLang	threads	0/2	2/5	1/5
SGLang, determ.	coffee	2/2	0/5	5/5
SGLang, determ.	crisis	2/2	0/5	5/5
SGLang, determ.	story	2/2	0/5	5/5
SGLang, determ.	tcp	2/2	0/5	5/5
SGLang, determ.	threads	2/2	0/5	5/5

Table 7: Restart recovery through the gateway, 10 trials per row. Exactly-once: every position received once, request complete. Same text: final text equals either failure-free reference. Recovery: kill to the gateway's commit of the next position, seconds, median (min–max).

Engine	Mode	T	Kill	Exactly-once	Same text	Recovery s
vLLM	default	0	25	10/10	0/10	107.6 (105.6–109.1)
vLLM	default	0	128	10/10	1/10	107.6 (106.6–108.6)
vLLM	default	0	230	10/10	1/10	108.4 (107.2–109.7)
vLLM	default	0.8	25	10/10	0/10	108.4 (107.1–109.7)
vLLM	default	0.8	128	10/10	1/10	181.8 (163.8–188.3)
vLLM	default	0.8	230	10/10	10/10	184.8 (158.3–187.8)
vLLM	batch-inv.	0	128	10/10	10/10	173.0 (121.2–177.3)
SGLang	default	0	25	10/10	2/10	135.5 (126.0–143.5)
SGLang	default	0	128	10/10	0/10	134.5 (129.1–142.0)
SGLang	default	0	230	10/10	1/10	130.7 (125.5–141.0)
SGLang	default	0.8	25	10/10	0/10	129.8 (122.5–141.5)
SGLang	default	0.8	128	10/10	0/10	132.1 (123.0–139.5)
SGLang	default	0.8	230	10/10	0/10	137.8 (130.6–192.1)
SGLang	determ.	0	128	10/10	10/10	178.8 (135.0–184.6)

Table 8: Failover between two machines (vLLM, kill after 128 positions, 10 trials per row). Replica 1 setup: its own compiled-kernel caches or copies of replica 0's, and whether its Python packages were identical to replica 0's. Recovery: kill to the gateway's commit of the next position, seconds, median (min–max).

Mode	T	Replica 1 setup	Exactly-once	Same text	Recovery s
default	0	own cache, packages differ	10/10	2/10	0.05 (0.05–0.39)
default	0.8	own cache, packages differ	10/10	10/10	0.05 (0.04–0.07)
batch-inv.	0	own cache, packages differ	10/10	0/10	0.08 (0.08–0.41)
batch-inv.	0	own cache, identical packages	10/10	0/10	0.08 (0.08–0.40)
batch-inv.	0	shared cache, identical packages	10/10	10/10	0.08 (0.08–0.40)

Table 9: Failure-free cost of the gateway: 30 requests direct to the engine and 30 through the gateway per row, alternating; medians, and the median of per-pair differences (Δ). Through the gateway, time to first token and per output token from the gateway's log, end to end at the client. Direct requests send text without asking for token ids, as a client without the gateway would.

			Time to first token (ms)			Time per output token (ms)			End to end, 256 tokens (s)		
Engine	fsync	Pairs	direct	gateway	Δ	direct	gateway	Δ	direct	gateway	Δ
vLLM	no	30	40.8	46.3	+5.4	33.11	33.29	+0.18	8.48	8.54	+0.05
vLLM	yes	30	40.9	49.5	+8.6	33.11	33.29	+0.18	8.48	8.54	+0.06
SGLang	no	30	70.1	70.5	+0.2	32.81	32.81	+0.00	8.44	8.44	+0.00
SGLang	yes	30	64.7	67.9	+0.3	32.81	32.81	+0.00	8.43	8.44	+0.01

Table 10: The anomaly fixes under concurrent load: output tokens per second at C requests in flight (mean of two repetitions; in parentheses, relative to the engine's default), median time per output token (range over levels and repetitions), and the number of distinct temperature-0 outputs per prompt across a whole run (range over the six prompts; repetition 1 / 2).

Config	C=1	C=4	C=16	C=32	TPOT ms	Outputs per prompt
vLLM default	30	115	435	856	33–37	5–8 / 5–9
vLLM batch-invariant	14 (0.47×)	58 (0.51×)	237 (0.54×)	488 (0.57×)	65–70	1 / 1
vLLM no APC	30 (1.00×)	115 (1.00×)	433 (1.00×)	845 (0.99×)	33–37	5–10 / 4–9
SGLang default	30	115	436	862	33–37	3–6 / 3–6
SGLang deterministic	14 (0.46×)	57 (0.49×)	233 (0.53×)	477 (0.55×)	67–71	1 / 1

Table 11: Recompute versus restore, vLLM, Llama 3.1 8B, A10G. Recompute: median prefill time. Restore columns: median seconds until the KV cache is on the GPU that will continue, and in parentheses recompute time divided by restore time (above 1: restore faster). Disk: read with a cold page cache, then host to GPU. Second machine: sender GPU to host, network, receiver host to GPU.

			Restore from, seconds (recompute / restore)			
L (tokens)	MB	Recompute s	Host memory	Local disk	2nd machine, 1 stream	2nd machine, 4 streams
512	67	0.137	0.005 (27.4)	0.19 (0.71)	0.118 (1.16)	0.064 (2.14)
1,024	134	0.270	0.010 (27.0)	0.39 (0.69)	0.236 (1.14)	0.128 (2.10)
2,048	268	0.496	0.020 (24.9)	0.78 (0.63)	0.473 (1.05)	0.257 (1.93)
4,096	537	1.027	0.040 (25.8)	1.58 (0.65)	0.946 (1.09)	0.513 (2.00)
8,192	1,074	2.206	0.080 (27.7)	3.15 (0.70)	1.891 (1.17)	1.026 (2.15)
16,384	2,147	5.029	0.159 (31.5)	6.27 (0.80)	3.783 (1.33)	2.052 (2.45)
32,768	4,295	12.544	0.319 (39.3)	12.56 (1.00)	7.566 (1.66)	4.105 (3.06)

Table 12: SIMULATION. Mixed workload, mean time between failures 1 hour. Ckpts: checkpoints per request at β=0.1. Overhead: failure-free completion time relative to recompute, in %, at $\beta = 0.1$ / 0.5 / 1. Recovery after a failure during decoding, mean / p99 seconds, at β=0.1 (reported after the run, outside the decision rule).

Policy	Ckpts	Overhead %			Recovery s
recompute	0.0	0.00	0.00	0.00	0.89 / 9.00
prompt-only	1.0	0.21	1.07	2.14	0.17 / 0.97
fixed-16	28.1	0.25	1.23	2.47	0.09 / 0.32
fixed-64	7.4	0.24	1.22	2.45	0.11 / 0.44
fixed-256	2.2	0.24	1.18	2.37	0.14 / 0.45
fixed-1024	1.1	0.22	1.11	2.22	0.16 / 0.69
adaptive-proposal	0.0	0.00	0.00	0.00	0.89 / 9.00
adaptive-remaining	0.0	0.00	0.00	0.00	0.89 / 9.00

Table 13: Cost of KV offloading while serving, 4,096-token prompts, 256 output tokens, range over two repetitions. TTFT: median time to first token. Relative throughput: output tokens/s over the mean of mode *none*.

	C=1			C=4	
Mode	TTFT ms	TPOT ms	Rel. tok/s	TPOT ms	Rel. tok/s
none	1028–1029	34.23–34.24	1.000	45.62–45.63	1.000
prompt	1082–1086	34.19	0.995–0.996	44.58	0.994–0.995
all	1032–1065	34.19	0.999–1.001	45.52–45.56	0.997

Table 14: Failover with a KV checkpoint, one request at a time (vLLM, kill after 128 positions, 5 trials per row, commit `1b978f2`). Text ok: the text the client concatenated equals the tokenizer's decoding of the delivered token ids, special tokens identified by id. Recovery: kill to the gateway's commit of the next position, s, median (min–max). TTFT: median time to first token on replica 0. KV loaded: increase of the standby's `vllm:kv_offload_load_bytes` during the trial, MB, median. Repeats removed: trials in which the engine repeated earlier text after the resume and the gateway removed it.

Prompt tokens	Mode	Exactly-once	Text ok	Recovery s	TTFT s	KV loaded, MB	Repeats removed
4,096	none	5/5	5/5	1.08 (1.08–1.12)	1.40	–	1
4,096	prompt	5/5	5/5	0.24 (0.21–0.59)	1.51	537	0
4,096	all	5/5	5/5	0.24 (0.21–0.56)	1.53	552	0
16,384	none	5/5	5/5	5.14 (5.10–5.14)	5.41	–	0
16,384	prompt	5/5	5/5	0.64 (0.64–0.67)	5.70	2,147	0
16,384	all	5/5	5/5	0.65 (0.63–0.67)	5.73	2,162	0

Table 15: Failover with C requests in flight (vLLM, 1,024-token prompts, 256 output tokens, replica 0 killed once request 0 has received 128 positions, 3 trials per row, one record per request, commit `1b978f2`). Columns as in Table 14; KV loaded is per trial (all requests together).

C	Mode	Requests	Exactly-once	Text ok	Recovery s	TTFT s	KV loaded, MB	Repeats removed
4	none	12	12/12	12/12	0.98 (0.30–1.20)	0.93	–	1
4	prompt	12	12/12	12/12	0.27 (0.12–0.48)	1.06	537	1
16	none	48	48/48	48/48	2.73 (0.31–4.48)	2.72	–	7
16	prompt	48	48/48	48/48	0.95 (0.25–1.07)	2.54	2,147	6
16	all	48	48/48	48/48	0.53 (0.29–0.72)	2.79	2,382	10
32	none	96	96/96	95/96	4.86 (0.33–15.20)	4.67	–	19
32	prompt	96	96/96	94/96	1.48 (0.42–8.00)	4.65	4,817	19

References

[1] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. Structured streaming: A declarative API for real-time applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[2] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. Lightweight asynchronous snapshots for distributed dataflows, 2015. arXiv:1506.08603.

[3] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[4] J. T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

[5] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002.

[6] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, 2025. `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.

[9] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. SpotServe: Serving generative large language models on preemptible instances. In *Proceedings of ASPLOS*, pages 1112–1127, 2024.

[10] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025.

[11] SGLang Team. Towards deterministic inference in SGLang and reproducible RL training. LMSYS Org blog, 2025. `https://lmsys.org/blog/2025-09-22-sglang-deterministic/`.

[12] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 46745–46771, 2024.

[13] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[14] John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.

[15] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.

[16] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.